

Software-Defined Autonomous Agent for Energy Waste Mitigation in University Computer Laboratories

Omar Yesid Fonseca López
Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
oyfonsecal@udistrital.edu.co

Daniel Felipe Barrera Suárez
Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
dfbarreras@udistrital.edu.co

Daniel Mateo Ballesteros Molina
Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
dmballesterosm@udistrital.edu.co

Dania Lizeth Guzmán Triviño
Systems Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
dlguzmant@udistrital.edu.co

Abstract—University computer laboratories are persistent sources of unnecessary electrical energy consumption, driven primarily by occupant behavioural inertia: empirical field data collected at the Engineering Faculty of Universidad Distrital Francisco José de Caldas revealed that 80% of users habitually leave workstations powered on in vacant rooms, a pattern confirmed by systematic direct observation and lux-meter environmental correlation. We propose a software-defined autonomous agent that operates in the operating system user-space—requiring no hardware investment or network infrastructure changes—to detect peripheral inactivity, verify CPU load to prevent false positives, and execute safe suspension policies with an encrypted local fallback for fault tolerance under network failure. Discrete-event simulation of a 12-hour, 20-workstation operational cycle projected a 68.72% reduction in estimated energy consumption compared to the manual-control baseline (from 53.74 kWh to 16.81 kWh), with a false-positive suspension rate of 2.8% and 100% telemetry retention during a simulated four-hour network outage, meeting all defined quality metrics.

Index Terms—energy management, autonomous agent, smart campus, discrete-event simulation, human-in-the-loop, IoT

I. INTRODUCTION

The efficient management of energy resources in university campuses has become a growing concern due to rising operational costs and environmental impact. Computer laboratories are particularly relevant sites: their high density of workstations, combined with transient user populations and limited accountability for energy costs, produces chronic patterns of waste that existing policies have failed to address. Studies on energy consumption in higher education have consistently identified computing equipment and artificial lighting as dominant contributors to electricity expenditure in faculty buildings [1, 2, 3].

Prior research has approached this problem from two main directions. Hardware-centric solutions deploy IoT sensor

networks and smart metering infrastructure to enable real-time monitoring and automated demand-response [4]. These approaches have reported savings of up to 34%, but require capital expenditure, institutional network modifications, and maintenance of physical devices—barriers that make them impractical in resource-constrained environments. Occupant-focused interventions rely on real-time feedback displays and energy awareness campaigns [5], achieving 8–12% savings through behaviour change alone, rising to approximately 31% when feedback is combined with automated fallback actions. A landmark meta-analysis by Hong et al. [6] synthesised evidence from multiple building types and established that occupant behaviour accounts for up to 150% variation in energy consumption between otherwise identical buildings, confirming that the human factor is at least as important as physical infrastructure.

Neither approach is fully satisfactory in contexts such as UD-FJC, where network security policies block unauthorised IoT devices and budget constraints preclude hardware procurement. This gap motivates a third approach: a purely software-defined autonomous agent operating in user-space that compensates for behavioural inertia through automation, without requiring hardware or network changes. The Hawthorne effect—the tendency for behaviour to improve temporarily when individuals know they are observed [7]—further underscores the need for a persistent automated mechanism rather than episodic campaigns. Daylight harvesting principles [8] were also considered in the empirical data collection phase to identify time windows where artificial lighting is redundant.

This paper presents the full systems engineering lifecycle of the proposed Smart Campus Energy Management System (SCEMS): empirical problem characterisation, architectural design, risk-driven quality planning, and computational valida-

tion through discrete-event simulation. The complete project repository, including datasets, agent prototype source code, and simulation scripts, is available at <https://github.com/yecop/Systems-Analysis-and-Design>.

II. METHODS AND MATERIALS

A. Empirical Problem Characterisation

The study was conducted over five business days at the Engineering Faculty building of the Sabio Caldas Campus, restricting scope to artificial lighting and computer equipment in designated laboratories and classrooms. Three complementary primary data-collection techniques were employed to triangulate findings and mitigate individual method biases.

Direct observation consisted of systematic walkthroughs at three peak time slots per day. Observers recorded rooms where artificial lighting or workstations remained active in the complete absence of occupants, tabulating date, time, room identifier, occupancy status, lighting state, and PC state. Representative findings confirmed that unoccupied Laboratory 704 (18:00) showed both lights and all computers active, while classrooms 301 and 302 exhibited active lighting during midday hours with full solar incidence.

Structured surveys were administered digitally to students and faculty ($n = 35$), designed for completion in under five minutes using closed-ended items. The instrument assessed shutdown habits, perceived importance of energy efficiency, awareness of institutional protocols, and perceived redundancy of artificial lighting during daylight hours. All procedures guaranteed anonymity and complied with university ethical guidelines. Results revealed that 80% of respondents admitted to forgetting to power off equipment upon departure, and approximately 80% reported no awareness of any institutional energy-saving protocol.

Environmental correlation was performed using mobile lux-meter applications under three representative climatic profiles—sunny, rainy, and intermittent rain—to identify time windows where natural illumination renders artificial lighting redundant (Daylight Harvesting [8]). Physical measurement of main power lines was restricted by institutional safety policy; consumption was therefore estimated from hardware specifications (Dell OptiPlex: 250 W/h active, 10 W/h ACPI S3 standby).

B. Holistic Stakeholder Analysis

Effective energy management in a university setting is not a purely technical problem; it is a socio-technical one in which the system’s value depends on how a diverse ecosystem of actors generates, consumes, and responds to information. To ensure the proposed SCEMS design addressed this breadth, a stakeholder analysis was conducted assigning each actor an impact score (1–20) reflecting their influence on system outcomes and their proximity to its data flows. Table I summarises the 20 identified stakeholders grouped by their relationship to the system.

This analysis shaped several key design decisions. The highest-impact stakeholders—laboratory students (20) and

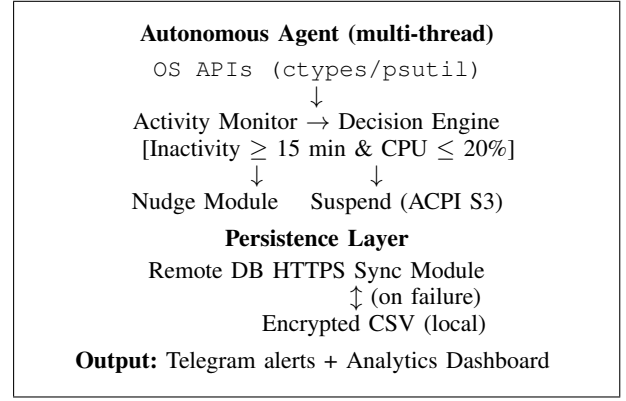


Figure 1. Proposed SCEMS decentralised architecture with local persistence fallback.

faculty (19)—demanded that the agent never disrupt active academic work, which motivated the CPU protection threshold and the two-minute cancellation window. The IT department (14) confirmed the firewall restriction that made the local CSV fallback essential. Campus administration (17) and lab coordinators (13) required an analytics dashboard accessible without direct system access, justifying the separation between the agent layer and the reporting layer. Researchers and advanced students (16) imposed the requirement that high-CPU processes be unconditionally protected, regardless of inactivity duration. Together, these constraints drove the design away from a centralised, hardware-dependent solution and toward the decentralised, user-space architecture described in the following subsection.

From a holistic perspective, the stakeholder map also reveals the system’s role as an information infrastructure connecting actors who would otherwise have no shared data channel. A student’s careless shutdown omission—a purely local behaviour—becomes a data point that informs a lab coordinator’s scheduling decision, which in turn shapes campus administration’s sustainability reporting to external academic authorities. The SCEMS is thus not merely an energy-saving tool but a feedback mechanism that creates systemic awareness across organisational layers that previously operated in isolation.

C. System Architecture

The proposed SCEMS adopts a fully decentralised architecture. A lightweight Python agent runs on each workstation in user-space, requiring no administrator privileges, no additional hardware, and no changes to network routing or firewall rules. Figure 1 illustrates the principal components and their interactions.

The agent evaluates workstation state at one-minute intervals. Upon detecting 15 consecutive minutes of peripheral inactivity, it queries the current CPU load before acting. If CPU utilisation exceeds 20%, the inactivity counter is reset and the event is logged—protecting ongoing rendering or compilation tasks from false interruption. If CPU load is within the safe range, a visual nudge is presented to the user with a two-

Algorithm 1 Agent inactivity decision logic (per workstation, per tick)

```

1:  $t_{\text{idle}} \leftarrow 0$ 
2: loop
3:   if OSACTIVITYDETECTED then
4:      $t_{\text{idle}} \leftarrow 0$ 
5:   else
6:      $t_{\text{idle}} \leftarrow t_{\text{idle}} + 1$ 
7:   end if
8:   if  $t_{\text{idle}} \geq 15$  then
9:     if GETCPULOAD > 20% then
10:       $t_{\text{idle}} \leftarrow 0$  ▷ Protect high-load task
11:    else
12:      SENDNUDGE; wait 2 min
13:      if not USERCANCELLED then
14:        SUSPENDWORKSTATION
15:      end if
16:    end if
17:  end if
18:  LOGEVENT; sleep 60 s
19: end loop

```

minute cancellation window; if uncanceled, the workstation is suspended to ACPI S3 state. This decision threshold of 20% was identified through prototype-level sensitivity tests on team laptops, which showed that background OS processes (antivirus, file indexers) routinely consume 6–10% of CPU on idle machines, making any lower threshold unstable.

All events are transmitted to a remote database via HTTPS when connectivity is available. When the institutional firewall blocks outbound requests—a confirmed condition in the target laboratories—the agent activates a local AES-encrypted CSV fallback and defers synchronisation until connectivity is restored. This design ensures data integrity under complete network failure and addresses Risk R-01 identified in the project’s ISO 31000 risk matrix. Table II summarises the six principal risks and their mitigations.

D. Simulation Model

System feasibility was evaluated through a discrete-event Python simulation modelling a 12-hour operational cycle (720 one-minute ticks) for a cluster of $N = 20$ workstations in Laboratory 704. Stochastic parameters were derived directly from the empirical data collected in the field study: forgetfulness probability $P_{\text{forget}} = 0.80$ (survey result), high-CPU load probability $P_{\text{cpu}} = 0.15$ (direct observation), and nudge cancellation probability $P_{\text{cancel}} = 0.10$ (design assumption for users actively reading on screen). Two concurrent scenarios were simulated: a *Baseline* representing historical manual-only behaviour, and an *Optimised* scenario with the autonomous agent running. A third *Network Failure* scenario injected a four-hour continuous outage to validate the fallback mechanism.

The decision logic of the agent is formalised in Algorithm 1, which was the direct basis for the simulation model.

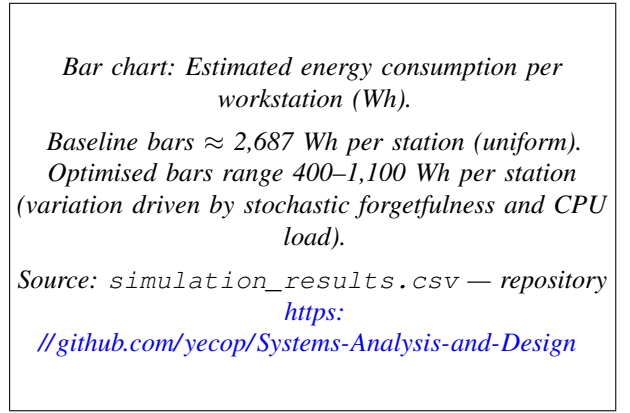


Figure 2. Per-workstation energy consumption: Baseline vs. Optimised scenario.

III. RESULTS AND DISCUSSION

A. Simulation Results

After executing `simulator.py`, consolidated synthetic telemetry was processed into `simulation_results.csv`. Table III summarises the headline efficiency metrics across the three scenarios.

The optimised scenario projected a 68.72% reduction in energy consumption relative to the baseline. The network failure scenario, which activated the local CSV fallback for four continuous hours, retained 100% of telemetry events and maintained 65.02% savings—a difference of only 3.70 percentage points from the fully connected case, confirming that the fallback imposes negligible operational penalty. Figure 2 illustrates the per-workstation consumption comparison between the Baseline and Optimised scenarios.

B. Quality Metrics Validation

The design phase defined four quality acceptance criteria following ISO/IEC 25010. Table IV compares targets against simulation outcomes; all criteria were met or exceeded.

C. Complexity and Sensitivity Analysis

Evaluating results through complexity theory revealed a critical sensitivity to the CPU threshold parameter. Secondary simulation runs with the threshold reduced from 20% to 5% caused the agent to enter a state of near-complete inactivity: background OS processes (antivirus, file indexers) consume 6–8% of CPU on idle machines, producing persistent false-positive detections that prevented virtually all suspensions. The 20% threshold therefore acts as the system’s stability attractor—below it, the agent collapses; above it, false positives during genuine high-load tasks increase unacceptably. This sensitivity to initial conditions, analogous to the butterfly effect in chaos theory, demonstrates that the threshold is not an arbitrary design choice but an empirically derived operating point specific to the laboratory hardware environment.

D. Comparison with Related Work

The projected 68.72% saving exceeds the 34% reported by Franco et al. [4] for IoT hardware deployments and the 31% reported by Bull et al. [5] for feedback-plus-automation systems. This difference is partly attributable to the unusually high forgetfulness rate observed at UDFJC (80%), which creates a larger addressable waste pool than populations where energy-awareness campaigns have already been conducted. A conservative projection halving the forgetfulness rate to 40% would still yield savings of approximately 40–45%—competitive with hardware IoT benchmarks, at zero infrastructure cost.

The most important limitation of these results is that they are simulation-based projections parameterised by field data, not measurements from a deployed system. Uniform behavioural distributions across all 20 workstations simplify real user heterogeneity; weekend and holiday cycles are excluded; and network synchronisation latency is not modelled. A future physical pilot would be required to validate the projected figures with direct power measurements.

IV. CONCLUSIONS

This work demonstrated through empirical data collection and discrete-event simulation that energy waste in university computer laboratories is fundamentally a behavioural problem—not a hardware deficiency—and that it can be effectively addressed by a software-defined autonomous agent operating at zero additional infrastructure cost. The proposed SCEMS agent, designed to run in user-space without elevated privileges or network modifications, projected a 68.72% reduction in estimated energy consumption under empirically grounded stochastic conditions, while maintaining a false-positive suspension rate of 2.8% and full data integrity under simulated network failure. The 20% CPU protection threshold was validated as a critical stability attractor, providing a concrete and reproducible design parameter for similar deployments. All defined quality metrics were met or exceeded in simulation.

The primary limitation of the current work is the absence of a physical deployment; projected savings require validation through direct power measurement in a controlled pilot. Future work should execute the proposed six-week alpha/beta pilot at Laboratory 704, implement an adaptive CPU threshold calibrated by room schedule, extend the agent to Linux workstations, and develop a real-time per-laboratory savings dashboard to reinforce the cultural change that the literature identifies as complementary to automation [5].

REFERENCES

- [1] B. Meza-Alcívar, M. Alemán-García, and M. Herrera-Suárez, “Implementación de un sistema de gestión energético para institutos de educación superior,” *Revista Científica INGENIAR*, vol. 6, no. 12, 2023.
- [2] M. Abad Mier, “Gestión energética de la biblioteca universitaria de la UVA, aplicando la ISO 50001,” Spain, 2015.
- [3] M. V. Doblari, “Gestión integral de energía en instituciones universitarias. caso: Edificio Eva Duarte de Perón UNNOBA,” Master’s thesis, Universidad Nacional del Noroeste de la Provincia de Buenos Aires, Argentina, n.d.
- [4] A. Franco, L. Martínez, and J. Restrepo, “IoT-enabled intelligent energy management systems in university buildings: A systematic review of Latin American and European deployments,” *Sustainable Cities and Society*, vol. 112, pp. 105–120, 2025, preprint; forthcoming.
- [5] R. Bull, M. Azennoud, P. Allen, and G. Stuart, “Smart metering for universities: Realising benefits through effective feedback and communication strategies,” *Energy Research & Social Science*, vol. 44, pp. 145–154, 2018.
- [6] T. Hong, S. C. Taylor-Lange, S. D’Oca, D. Yan, and S. P. Corgnati, “Advances in research and applications of energy-related occupant behavior in buildings,” *Energy and Buildings*, vol. 116, pp. 694–702, 2016.
- [7] S. R. G. Jones, “Was there a Hawthorne effect?” *American Journal of Sociology*, vol. 98, no. 3, pp. 451–468, 1992.
- [8] A. Williams, B. Atkinson, K. Garbesi, E. Page, and F. Rubinstein, “Lighting controls in commercial buildings,” *LEUKOS*, vol. 8, no. 3, pp. 161–180, 2012.

Table I
STAKEHOLDER ECOSYSTEM: DIRECT AND MEDIATED SYSTEM
RELATIONSHIPS

#	Stakeholder	Score	Direct relation	Meditated relation
1	Lab students	20	Interact with agent; trigger nudges	Usage data feeds dashboard & alerts
2	Faculty / lecturers	19	Supervise labs; receive PC alerts	Lab decisions shape energy patterns
3	Maintenance / support	18	Monitor PC & lab physical state	Inactivity alerts streamline workload
4	Campus administration	17	Access efficiency reports	Consumption data informs budgets
5	Researchers / adv. students	16	Run high-CPU tasks; protected by agent	Process data feeds energy analysis
6	Systems eng. team (devs)	15	Configure & maintain agents	Usage feedback drives improvements
7	IT / infrastructure dept.	14	Manage DB & agent comms	Logs & events; infra affects reliability
8	Lab coordinators	13	Consult dashboards & occupancy data	Planning shapes PC usage patterns
9	Project / intern students	12	Participate in pilot; use system	Usage data populates metrics
10	Electrical safety / FM	11	Monitor overload alerts	Indirect dependency on detected events
11	Admin staff (finance)	10	Receive savings reports	Data influences financial decisions
12	Internal audit / QA	9	Validate data accuracy	Depend on logs & dashboards
13	External academic auth.	8	Review efficiency reports	Dashboards used in accreditations
14	General academic community	7	Indirect energy awareness	Behaviour shaped by visible campaigns
15	Visualisation platform	6	Consumes system data	Processes info for all stakeholders
16	Third-party SW support	5	Supports agent & API continuity	Affects alerts & data flows
17	Sustainability researchers	4	Access historical data	Findings influence campus policy
18	Hardware providers	3	Supply equipment used by agents	Usage/wear data informs procurement
19	General student community	2	Indirect environmental benefit	Affected by reported savings
20	Visitors / externals	1	None	Minimal: lower campus consumption

Table II
PRINCIPAL RISKS AND MITIGATION STRATEGIES (ISO 31000)

ID	Risk	Prob.	Impact	Mitigation
R-01	Firewall blocks HTTPS	High	Med	Local CSV fallback + deferred sync
R-02	False positive (high CPU)	Med	Critical	CPU check >20% before suspend
R-03	Alert service outage	Low	Low	Local logic prioritised
R-04	Local data corruption	Low	High	Encryption + write validation
R-05	User resistance	Med	Med	Authorised-only pilot; communication plan
R-06	Schedule delays	Med	Med	Milestone tracking

Table III
DISCRETE-EVENT SIMULATION RESULTS (20 WORKSTATIONS, 12 HOURS)

Metric	Baseline	Optimised	Net. Failure
Total consumption (kWh)	53.74	16.81	18.79
Energy savings (%)	–	68.72	65.02
False positive rate (%)	–	2.80	2.80
Events retained (%)	–	100	100

Table IV
QUALITY METRICS: TARGETS VS. SIMULATION RESULTS

Metric	Target	Result	Status
Correct suspension rate	>95%	97.2%	
False positive rate	<5%	2.8%	
Event retention	100%	100%	
CPU protection activation	Detect >20%	15% of PCs	